ktunotes.in

GET IT ON Google Play

KTU
NOTES
The learning companion.

KTU STUDY MATERIALS | SYLLABUS | LIVE NOTIFICATIONS | SOLVED QUESTION PAPERS

Website: www.ktunotes.in

Web Servers: Introduction, HTTP Transactions, Multitier Application Architecture, Client-Side Scripting versus Server-Side Scripting, Accessing Web Servers.

Server Side Scripting with Node.js: Getting to know node, node.js changed JavaScript forever, features of node, when to use and not use node, asynchronous callbacks, the NoSql movement, node and MongoDB in the wild, Hello World in Node, package.json, modules, *Built-in Modules:* FS Module, HTTP Module, Events; Node Package Manager(npm), web server using http, node.js with express, middleware, routing in express, CRUD operations in express, web server using express, making it live on Heroku

Node.js with MongoDB: basics of MongoDB, MongoDB CRUD Operations, Building a data model with MongoDB and Mongoose, Defining simple mongoose schemas, build node express app with MongoDB

Ktunotes.in

Web server is a computer where the web content is stored. Basically web server is used to host the web sites but there exists other web servers also such as gaming, storage, FTP, email etc.

Web site is collection of web pages while web server is a software that respond to the request for web resources.

Web Server Working

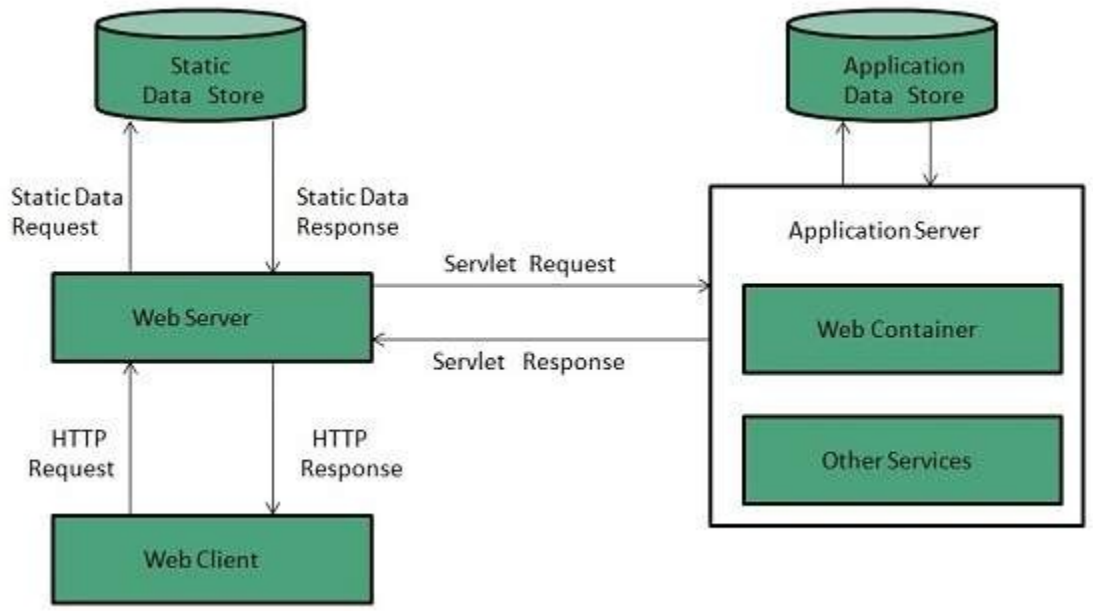
Web server respond to the client request in either of the following two ways:

Sending the file to the client associated with the requested URL.

Generating response by invoking a script and communicating with database

internet_technologies_tutorial

DOWNLOADED FROM KTUNOTES.IN



Example

Apache HTTP Server

Internet Information Services (IIS)

Sun Java System Web Server

Multitier Application Architecture

Distributed Network: It is a network architecture, where the components located at network computers coordinate and communicate their actions only by passing messages. It is a collection of multiple systems situated at different nodes but appears to the user as a single system.

It provides a single data communication network which can be managed separately by different networks.

An example of Distributed Network– where different clients are connected within LAN architecture on one side and on the other side they are connected to high-speed switches along with a rack of servers containing service nodes.

Client-Server Architecture: It is an architecture model where the client (one program) requests a service from a server (another program) **i.e.** It is a request-response service provided over the internet or through an intranet.

DOWNLOADED FROM KTUNOTES.IN

In this model, **Client** will serve as one set of program/code which executes a set of actions over the network. While **Server**, on the other hand, is a set of another program, which sends the result sets to the client system as requested.

- In this, client computer provides an interface to an end user to request a service or a resource from a server and on the other hand server then processes the request and displays the result to the end user.
- An example of Client-Server Model– an ATM machine. A bank is the server for processing the application within the large customer databases and ATM machine is the client having a user interface with some simple application processing.

Database: It is a collection of information in an organized way so that it can be easily accessed, managed and updated.

- Examples of Database – MySQL, SQL Server, and Oracle Database are some common Db's.

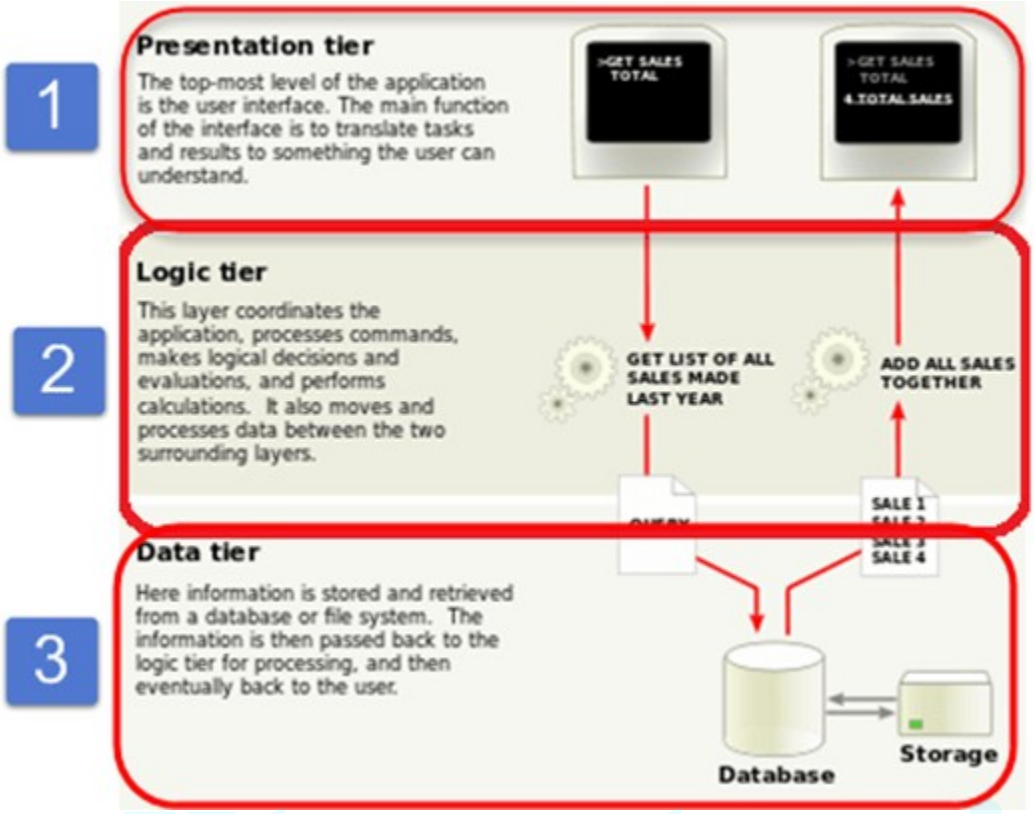
3-Tier Architecture

By looking at the below diagram, you can easily identify that **3-tier architecture** has three different layers.

- Presentation layer
- Business Logic layer
- Database layer

ktunotes.in

DOWNLOADED FROM KTUNOTES.IN



Ktunotes.in

Advantages and Disadvantages of Multi-Tier Architectures

Advantages	Disadvantages
• Scalability	• Increase in Effort
• Data Integrity	• Increase in Complexity
• Reusability	
• Reduced Distribution	
• Improved Security	

Client-Side Scripting versus Server-Side Scripting

Difference between client-side scripting and server-side scripting :

DOWNLOADED FROM KTUNOTES.IN

Client-side scripting	Server-side scripting
1)Source code is visible to the user.	1)Source code is not visible to the user because its output of server-sideside is an HTML page.
2)It usually depends on the browser and its version.	2)In this any server-side technology can be used and it does not depend on the client.
3)It runs on the user's computer.	3)It runs on the webserver.
4)There are many advantages linked with this like faster. response times, a more interactive application.	4)The primary advantage is its ability to highly customize, response requirements, access rights based on user.
5)It does not provide security for data.	5)It provides more security for data.
6)It is a technique used in web development in which scripts run on the client's browser.	6)It is a technique that uses scripts on the webserver to produce a response that is customized for each client's request.
7)HTML, CSS, and javascript are used.	7)PHP, Python, Java, Ruby are used.

Ktunotes.in

What is Node.js?

- Node.js is an open source server environment
- Node.js is free
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- Node.js uses JavaScript on the server

Features of Node js

DOWNLOADED FROM KTUNOTES.IN

Here is how PHP or ASP handles a file request:

1. Sends the task to the computer's file system.
2. Waits while the file system opens and reads the file.
3. Returns the content to the client.
4. Ready to handle the next request.

Here is how Node.js handles a file request:

1. Sends the task to the computer's file system.
2. Ready to handle the next request.
3. When the file system has opened and read the file, the server returns the content to the client.

Node.js eliminates the waiting, and simply continues with the next request.

Node.js runs single-threaded, non-blocking, asynchronous programming, which is very memory efficient.

Asynchronous Programming and Callbacks

Ktunotes.in

Computers are asynchronous by design.

Asynchronous means that things can happen independently of the main program flow.

In the current consumer computers, every program runs for a specific time slot and then it stops its execution to let another program continue their execution. This thing runs in a cycle so fast that it's impossible to notice. We think our computers run many programs simultaneously, but this is an illusion (except on multiprocessor machines).

Programs internally use interrupts, a signal that's emitted to the processor to gain the attention of the system.

Let's not go into the internals of this now, but just keep in mind that it's normal for programs to be asynchronous and halt their execution until they need attention, allowing the computer to execute other things in the meantime. When a program is waiting for a response from the network, it cannot halt the processor until the request finishes.

Normally, programming languages are synchronous and some provide a way to manage asynchronicity in the language or through libraries. C, Java, C#, PHP, Go, Ruby, Swift, and Python are all synchronous by default. Some of them handle async operations by using threads, spawning a new process.

JavaScript is **synchronous** by default and is single threaded. This means that code cannot create new threads and run in parallel.

DOWNLOADED FROM KTUNOTES.IN

Callbacks

You can't know when a user is going to click a button. So, you define an event handler for the click event. This event handler accepts a function, which will be called when the event is triggered:

JS

copy

```
document.getElementById('button').addEventListener('click', () => {

  //item clicked

})
```

This is the so-called callback.

Ktunotes.in

A callback is a simple function that's passed as a value to another function, and will only be executed when the event happens. We can do this because JavaScript has first-class functions, which can be assigned to variables and passed around to other functions (called higher-order functions)

It's common to wrap all your client code in a load event listener on the window object, which runs the callback function only when the page is ready:

```
window.addEventListener('load', () => {

  //window loaded

  //do what you want

})
```

Callbacks are used everywhere, not just in DOM events.

One common example is by using timers:

DOWNLOADED FROM KTUNOTES.IN

```
setTimeout(() => {

  // runs after 2 seconds

}, 2000)
```

XHR requests also accept a callback, in this example by assigning a function to a property that will be called when a particular event occurs (in this case, the state of the request changes):

```
const xhr = new XMLHttpRequest()

xhr.onreadystatechange = () => {

  if (xhr.readyState === 4) {

    xhr.status === 200 ? console.log(xhr.responseText) : console.error('error')

  }

}

xhr.open('GET', 'https://yoursite.com')

xhr.send()
```

Ktunotes.in

Handling errors in callbacks

How do you handle errors with callbacks? One very common strategy is to use what Node.js adopted: the first parameter in any callback function is the error object: error-first callbacks

If there is no error, the object is null. If there is an error, it contains some description of the error and other information.

```
fs.readFile('/file.json', (err, data) => {

  if (err) {

    //handle error

    console.log(err)
```

DOWNLOADED FROM KTUNOTES.IN

```
    return
  }

  //no errors, process data
  console.log(data)
})
```

Timer Functions

setTimeout() can be used to schedule code execution after a designated amount of milliseconds.

setImmediate() will execute code at the end of the current event loop cycle. This code will execute after any I/O operations in the current event loop and before any timers scheduled for the next event loop.

Ktunotes.in

Node.js Modules

Built-in Modules

Node.js has a set of built-in modules which you can use without any further installation.

Include Modules

To include a module, use the require() function with the name of the module:

```
var http = require('http');
```

Now your application has access to the HTTP module, and is able to create a server:

```
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

DOWNLOADED FROM KTUNOTES.IN

Node.js File System Module

The Node.js file system module allows you to work with the file system on your computer.

To include the File System module, use the require() method:

```
var fs = require('fs');
```

Common use for the File System module:

- Read files
- Create files
- Update files
- Delete files
- Rename files

Read Files

The fs.readFile() method is used to read files on your computer.

```
var http = require('http');

var fs = require('fs');

http.createServer(function (req, res) {

  fs.readFile('demofile1.html', function(err, data) {

    res.writeHead(200, {'Content-Type': 'text/html'});

    res.write(data);

    return res.end();

  });


}).listen(8080);
```

Create Files

The File System module has methods for creating new files:

Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN

- fs.appendFile()
- fs.open()
- fs.writeFile()

The fs.appendFile() method appends specified content to a file. If the file does not exist, the file will be created:

Update Files

The File System module has methods for updating files:

- fs.appendFile()
- fs.writeFile()

The fs.appendFile() method appends the specified content at the end of the specified file:

Node.js NPM

NPM is a package manager for Node.js packages, or modules if you like.

Ktunotes.in

www .npmjs.com hosts thousands of free packages to download and use.

The NPM program is installed on your computer when you install Node.js

A package in Node.js contains all the files you need for a module.

Modules are JavaScript libraries you can include in your project.

Node.js Events

Every action on a computer is an event. Like when a connection is made or a file is opened.

Objects in Node.js can fire events, like the readStream object fires events when opening and closing a file:

Events Module

Node.js has a built-in module, called "Events", where you can create-, fire-, and listen for- your own events.

DOWNLOADED FROM KTUNOTES.IN

To include the built-in Events module use the require() method. In addition, all event properties and methods are an instance of an EventEmitter object. To be able to access these properties and methods, create an EventEmitter object:

HTTP Module

The HTTP module provides a way of making Node.js transfer data over HTTP (Hyper Text Transfer Protocol).

The syntax for including the HTTP module in your application:

var http = require('http');

HTTP Properties and Methods

Method	Description
createClient()	Deprecated. Creates a HTTP client
createServer()	Creates an HTTP server
get()	Sets the method to GET, and returns an object containing the user's request
globalAgent	Returns the HTTP Agent
request()	Returns an object containing the user's request

Ktunotes.in

node.js with express

DOWNLOADED FROM KTUNOTES.IN

Express is a minimal and flexible Node.js web application framework that provides a robust set of features to develop web and mobile applications. It facilitates the rapid development of Node based Web applications. Following are some of the core features of Express framework −

- Allows to set up middlewares to respond to HTTP Requests.
- Defines a routing table which is used to perform different actions based on HTTP Method and URL.
- Allows to dynamically render HTML Pages based on passing arguments to templates.

Hello world Example

Following is a very basic Express app which starts a server and listens on port 8081 for connection. This app responds with **Hello World!** for requests to the homepage. For every other path, it will respond with a **404 Not Found.**

```
var express = require('express');
var app = express();

app.get('/', function (req, res) {
  res.send('Hello World');
})

var server = app.listen(8081, function () {
  var host = server.address().address
  var port = server.address().port

  console.log("Example app listening at http://%s:%s", host, port)
})
```

Save the above code in a file named server.js and run it with the following command.

$ node server.js

routing in express

Routing refers to how an application's endpoints (URIs) respond to client requests

You define routing using methods of the Express app object that correspond to HTTP methods; for example, app.get() to handle GET requests and app.post to handle POST requests. For a full list, see app.METHOD. You can also use app.all() to handle all HTTP methods and app.use() to specify middleware as the callback function

These routing methods specify a callback function (sometimes called "handler functions") called when the application receives a request to the specified route (endpoint) and HTTP method. In

DOWNLOADED FROM KTUNOTES.IN

other words, the application “listens” for requests that match the specified route(s) and method(s), and when it detects a match, it calls the specified callback function.

In fact, the routing methods can have more than one callback function as arguments. With multiple callback functions, it is important to provide next as an argument to the callback function and then call next() within the body of the function to hand off control to the next callback.

The following code is an example of a very basic route.

```
const express = require('express')
const app = express()

// respond with "hello world" when a GET request is made to the homepage
app.get('/', (req, res) => {
  res.send('hello world')
})
```

CRUD operations in express

ktunotes.in

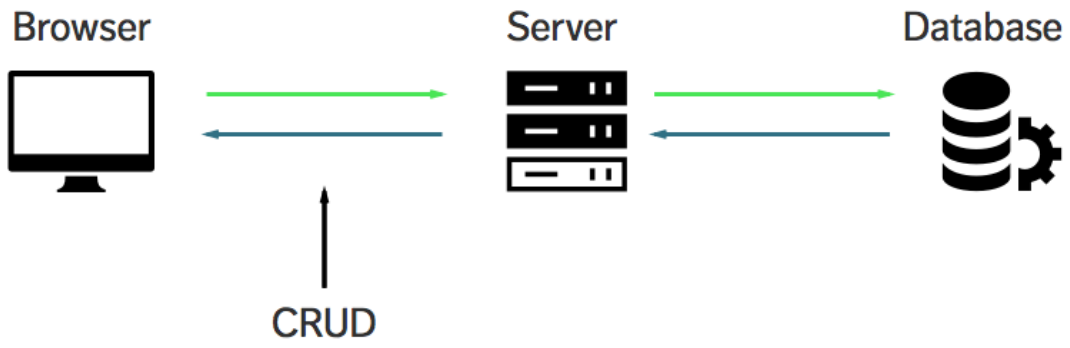
Express is a framework for building web applications on top of Node.js. It simplifies the server creation process that is already available in Node. In case you were wondering, Node allows you to use JavaScript as your server-side language.

MongoDB is a database. This is the place where you store information for your websites (or applications).

CRUD is an acronym for Create, Read, Update and Delete. It is a set of operations we get servers to execute (POST, GET, PUT and DELETE requests respectively). This is what each operation does:

- **Create (POST)** - Make something
- **Read (GET)**- Get something
- **Update (PUT)** - Change something
- **Delete (DELETE)**- Remove something

DOWNLOADED FROM KTUNOTES.IN



Ktunotes.in

DOWNLOADED FROM KTUNOTES.IN